

Take-Home Assignment — AI Agent Engineer

What this is

Build a conversational agent that handles appointment booking for a small business over **Telegram**, checking and booking real availability in **Google Calendar**. We want to see how you wire independent integrations together into a working agent — not just prompt an LLM (Large Language Model).

Effort and timeline

- **Expected effort: ~6–10 hours.** You have **3 days** from receipt — that window is for scheduling flexibility, not a signal to build more. A focused, correct, well-scoped submission beats a sprawling one.
- Partial completion is expected and fine. We evaluate how far you got and how well, milestone by milestone (see "How we evaluate"). Build in the order listed and get each piece solid before the next.

The task

A user messages a Telegram bot to book an appointment. The agent should:

1. Understand the request (greeting, general question, or a booking intent).
2. For a booking, **check Google Calendar availability** for the requested time.
3. If the slot is free → confirm details, then **book it** (create the calendar event) and confirm to the user.
4. If the slot is **not** free → **suggest the nearest available alternative**, and on the user's confirmation, book that one. ("Nearest" means the soonest available 30-minute slot at or after the requested time, on the same business day; if none remain that day, say so rather than rolling to the next day.)
5. **Collect the patient's email during the conversation and send them an email confirmation** once the appointment is booked — in addition to the Telegram confirmation message. How you send the email is your choice.
6. Answer basic business questions from the info below.

Required conversation to support (this exact shape will be tested):

User: "Can I book Monday at 2pm?" Agent: (2pm is taken) "2pm Monday isn't available. The nearest opening is 3pm — shall I book that?" User: "yes" Agent: (books 3pm) "Done — you're booked for Monday 3:00pm. Anything else?"

Note that "yes" must resolve to the slot you proposed. Handle the multi-turn state correctly.

Business info to use (fictional business)

- **Name:** BrightCare Clinic.
- **Hours:** Monday–Friday, 09:00–18:00. Closed weekends.
- **Appointment length:** 30 minutes.
- **A few FAQs the agent should answer:** location is 12 Orchard Rd; walk-ins not accepted (appointment only); to cancel, the user should message the clinic; parking is available on-site.

Required integrations (all with your own accounts — we provide nothing)

Choose and justify your integration approach. We care about your reasoning, not just that it works — explain your key choices in the README.

1. **Telegram Bot** — create your own bot via @BotFather so users can message it. How you connect it and how the bot receives messages is your call; be ready to justify the approach.
2. **Google Calendar** — use **your own calendar**. Either auth path is fine:
 - a **service account** with your calendar shared to the service-account email, or
 - **OAuth** (Open Authorization) with yourself as a test user (an app in "Testing" status works without Google verification). We score a working read + write, not which method you chose.
3. **LLM** — use **your own API key** (Google Gemini has a free tier; any provider is fine).

Functional requirements

- Multi-turn conversation with state carried across turns (the "yes → proposed slot" case above).
- Check availability against the real calendar before proposing or booking.
- Book by creating a real event; the booking must land in your calendar (you'll show this in the demo).
- **Collect the patient's email as part of the conversation, carry it in state, and send a confirmation email to that address after booking.** The patient must actually receive the email — **verify it arrives in a real inbox; do not assume a successful API response means the email was sent.** Show the received email in your demo.
- Handle at least these edge cases sensibly: requested time outside business hours; no availability on the requested day; an off-topic or unclear message; a missing or malformed email address.

What to submit

1. A **Git repository** (link) with your code. Should have Git commit history.
2. A **README** covering: how to run it, how you set up each integration, your design in a few sentences, what you'd improve with more time, and any assumptions.
3. A **2–3 minute screen-recording** demoing the full flow end to end, including the booking appearing in your Google Calendar **and the confirmation email arriving in the patient's inbox.**

Notes

- Language and framework are your choice.
- Using AI coding assistants is allowed and expected — but you should fully understand your submission. We may ask you to walk through and justify parts of it.
- Do **not** hardcode secrets; use environment variables. Include a `.env.example`.
- Optional bonus (only if the core is solid): timezone handling, deployment to a cloud endpoint, tests, graceful handling of concurrent bookings.